

SUPPLY MANAGEMENT SYSTEM — Functional Specification Document

FSD Addendum 34 — Route Classification, Lead Time & Production-to-Delivery Integration

Property	Value
Document ID	SMS-FSD-ADD-034
Version	1.0
Date	2026-10-03
Status	Draft
Author	System Architect
Classification	Internal — Development
Depends On	SMS-FSD-ADD-031 (Manufacturing Flow Corrections), SMS-FSD-ADD-033 (Fulfillment Routes & SO Delivery)
Architecture	.NET 8 / EF Core 8 / SQL Server 2022 / React 18
Change Items	6 changes (C1-C6)
Impact	4 migrations, 5 modified entities, 1 new table, 4 new/modified UI panels, 1 new service, 1 extended event handler

Table of Contents

- 1. [Purpose & Scope](#)
- 2. [Existing Production Order Review](#)
- 3. [C1 — Route Category & Manufacturing Seed Routes](#)
- 4. [C2 — Route-Based Product Classification](#)
- 5. [C3 — Lead Time Management on Products](#)
- 6. [C4 — Per-Line Lead Time Calculation with BOM Awareness](#)
- 7. [C5 — SO Confirmation: Split by Route Category](#)
- 8. [C6 — Production Completion → Delivery Auto-Creation](#)

9. Database Migrations
 10. API Changes
 11. UI Wireframes & Specifications
 12. Business Rules
 13. Test Scenarios
 14. Development Phases
-

1. Purpose & Scope

This addendum extends the Fulfillment Route system (Addendum 33) with a route category that classifies products by their fulfillment strategy, replaces boolean production flags with route-driven classification, introduces lead time management at product and line-item level, and defines the production-to-delivery bridge — how a completed Production Order automatically creates a Delivery Order when linked to a Sale Order.

1.1 Design Principles

- **Route as Single Classifier** — The fulfillment route's `route_category` replaces `is_manufacturable` and similar flags as the authoritative mechanism for determining product behavior. Existing flags remain as read-only filter aids.
- **Single Route per Variant** — Each product variant carries exactly one fulfillment route. SO line override provides the escape hatch when the default route doesn't fit a specific order.
- **Production-then-Delivery** — For manufacturing-category products on a Sale Order, the system creates a Production Order at SO confirmation and defers delivery until production completes (FGR confirmed). Non-manufacturing products continue with immediate delivery creation (Addendum 33 flow).
- **Lead Time Transparency** — Lead time is broken into measurable components at product level and available as an on-demand calculation at every pipeline stage (Inquiry, Quotation, Sale Order).
- **BOM Procurement Deferred** — Raw material procurement workflows (creating Purchase Orders from BOM shortages) are handled by the existing Supply Requirement Engine (Addendum 31 C3) and will be further refined in a future addendum.

1.2 Changes Summary

Change ID	Title	Category
C1	Route Category & Manufacturing Seed Routes	Extend entity — add route_category to FulfillmentRoutes
C2	Route-Based Product Classification	Modify behavior — BOM tab, SO confirmation, filters driven by route
C3	Lead Time Management on Products	New feature — lead time component columns + org defaults + UI tab
C4	Per-Line Lead Time Calculation with BOM Awareness	New service — ILeadTimeCalculator with recursive BOM traversal
C5	SO Confirmation: Split by Route Category	Modify service — manufacturing lines create Production Order, stock lines create Delivery
C6	Production Completion → Delivery Auto-Creation	Extend event handler — FGR triggers delivery for SO-linked Production Orders

2. Existing Production Order Review

The following Production Order implementation (Addendum 30, corrected by Addendum 31) is reviewed for compatibility with the changes in this addendum.

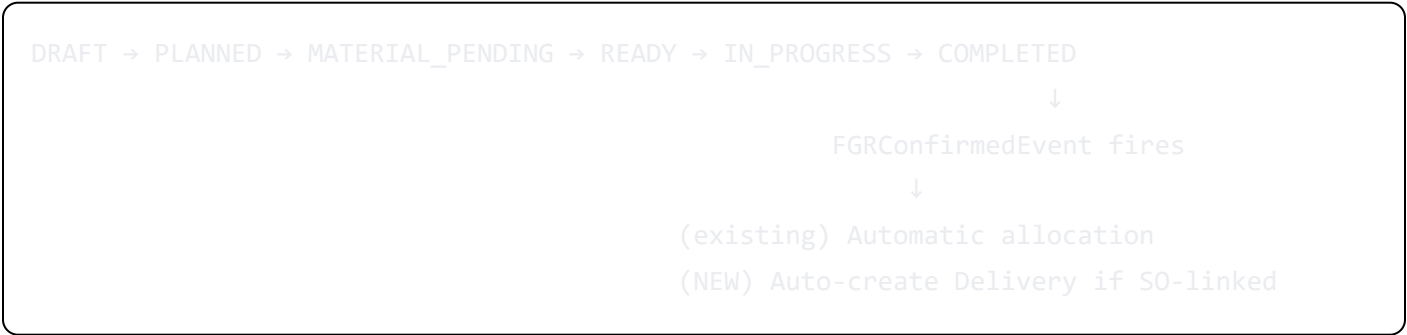
2.1 Current Production Order Schema (material.ProductionOrders)

Column	Status	Notes
production_order_id	✓ No change	PK
org_id	✓ No change	Tenant discriminator
product_id	✓ No change	Product being manufactured
variant_id	✓ No change	Specific variant being manufactured
planned_qty	✓ No change	Quantity to produce
actual_qty	✓ No change	Quantity actually produced (updated at FGR)
status	✓ No change	State machine: Draft → Planned → MaterialPending → Ready → InProgress → Completed
production_warehouse_id	✓ No change	Where production happens
planned_start_date	✓ No change	
planned_end_date	✓ No change	
sale_order_id	✓ Already exists (ADD-031 C7)	FK → demand.SaleOrders. NULL for standalone production.
sale_order_line_id	✓ Already exists (ADD-031 C7)	FK → demand.SaleOrderLines. NULL for standalone production.
fulfillment_route_id	⚠ NEW — Required by this addendum	Resolved from SO line at creation time. Used to create delivery after FGR.

2.2 Required Changes to Production Orders

#	Change	Reason
1	Add fulfillment_route_id column (INT NULL FK)	When the Production Order completes, the system needs to know which fulfillment route to use for the delivery. Resolved from the SO line's effective route at PO creation time. NULL for standalone POs.
2	Extend FGRConfirmedEventHandler	Currently does automatic allocation (BR-C10-02, Addendum 31). Must also trigger delivery creation when PO has a sale_order_id.
3	No change to Production Order state machine	The existing Draft → Planned → MaterialPending → Ready → InProgress → Completed flow is unaffected.
4	No change to BOM Explosion or PMR	The manufacturing process itself is unchanged.
5	No change to Supply Requirement Engine	Raw material procurement via Draft POs (Addendum 31 C3) continues as-is.

2.3 Production Order Status Reference



2.4 What Is NOT Changing

Component	Status
BOM Explosion (BOMExplosionService)	✓ Unchanged
Production Material Requirements (PMR)	✓ Unchanged
Material Issue workflow	✓ Unchanged
Quality Inspection	✓ Unchanged
Finished Goods Receipt (FGR)	✓ Unchanged — event handler extended
Supply Requirement Engine (Draft PO flow)	✓ Unchanged
Material Availability Tab (Addendum 31 C8)	✓ Unchanged
Consolidated Purchase Required (Addendum 31 C9)	✓ Unchanged

3. C1 — Route Category & Manufacturing Seed Routes

3.1 Overview

Add a `route_category` column to `logistics.FulfillmentRoutes` (created in Addendum 33). The category classifies the fulfillment strategy and determines system behavior — whether the product is fulfilled from stock (delivery at SO confirmation) or requires production first (delivery deferred to production completion).

3.2 Route Categories

Category	Description	SO Confirmation Behavior	Delivery Trigger
STOCK	Product fulfilled from existing inventory	Create Delivery Order immediately	SO confirmation
MANUFACTURE	Product must be manufactured before delivery	Create Production Order; defer delivery	FGR completion
BUY	Product must be procured from supplier before delivery (future)	Reserved for future addendum	Reserved
DROPSHIP	Product ships directly from supplier to customer (future)	Reserved for future addendum	Reserved

 **NOTE:** `BUY` and `DROPSHIP` categories are defined for schema completeness but not activated in this addendum. All non-MANUFACTURE routes default to `STOCK` behavior.

3.3 Schema Change — FulfillmentRoutes

Modify: `logistics.FulfillmentRoutes` (from Addendum 33)

FulfillmentRoutes — New Column (ADD)

Column	Type	Nullable	Default	Description
route_category	NVARCHAR(20)	NOT NULL	'STOCK'	Classification: STOCK, MANUFACTURE, BUY, DROPSHIP

Constraint:

```
sql
CONSTRAINT CK_FulfillmentRoutes_Category
CHECK (route_category IN ('STOCK', 'MANUFACTURE', 'BUY', 'DROPSHIP'))
```

3.4 Updated Seed Routes

Existing seed routes (Addendum 33) — updated with category:

Code	Name	Category	Steps
PICK_ONLY	Pick Only	STOCK	PICK → GOODS_ISSUE
PICK_AND_SHIP	Pick & Ship	STOCK	PICK → GOODS_ISSUE → SHIP
PICK_PACK_SHIP	Pick, Pack & Ship	STOCK	PICK → PACK → GOODS_ISSUE → SHIP

New manufacturing seed routes:

Code	Name	Category	Steps
MFG_PICK_SHIP	Manufacture → Pick & Ship	MANUFACTURE	PICK → GOODS_ISSUE → SHIP
MFG_PICK_PACK_SHIP	Manufacture → Pick, Pack & Ship	MANUFACTURE	PICK → PACK → GOODS_ISSUE → SHIP

 **CRITICAL:** Manufacturing routes carry the same delivery steps as their stock counterparts. The manufacturing process itself (BOM explosion, material issue, production) is handled by the Production Order — not by the fulfillment route. The route_category tells the SO confirmation service to create a Production Order FIRST. Once production completes, the route's steps drive the Delivery Order exactly like a stock route would.

3.5 Seed Route Insert (Per Org — Tenant Provisioning)

sql


```
-- New manufacturing routes (added to existing seed provisioning)

INSERT INTO logistics.FulfillmentRoutes
  (org_id, code, name, description, route_category, is_default, is_active, is_system,
   requires_packing, requires_shipping, display_order)
VALUES
  (@orgId, 'MFG_PICK_SHIP', 'Manufacture → Pick & Ship',
   'Product manufactured then picked and shipped without packing.',
   'MANUFACTURE', 0, 1, 1, 0, 1, 40),
  (@orgId, 'MFG_PICK_PACK_SHIP', 'Manufacture → Pick, Pack & Ship',
   'Product manufactured then picked, packed and shipped.',
   'MANUFACTURE', 0, 1, 1, 1, 1, 50);

-- Steps for MFG_PICK_SHIP
INSERT INTO logistics.FulfillmentRouteSteps
  (fulfillment_route_id, org_id, step_code, step_order, is_mandatory)
VALUES
  (@mfgPickShipId, @orgId, 'PICK', 1, 1),
  (@mfgPickShipId, @orgId, 'GOODS_ISSUE', 2, 1),
  (@mfgPickShipId, @orgId, 'SHIP', 3, 1);

-- Steps for MFG_PICK_PACK_SHIP
INSERT INTO logistics.FulfillmentRouteSteps
  (fulfillment_route_id, org_id, step_code, step_order, is_mandatory)
VALUES
  (@mfgPickPackShipId, @orgId, 'PICK', 1, 1),
  (@mfgPickPackShipId, @orgId, 'PACK', 2, 1),
  (@mfgPickPackShipId, @orgId, 'GOODS_ISSUE', 3, 1),
  (@mfgPickPackShipId, @orgId, 'SHIP', 4, 1);
```

3.6 Route Category on Route Editor UI

The Route Editor (Addendum 33 §10.2) gains a Category dropdown at the top:

Category: [STOCK ▼] ← STOCK, MANUFACTURE (BUY, DROPSHIP disabled)

When category = MANUFACTURE, the editor shows a note: *"Products with this route will trigger a Production Order at Sale Order confirmation. Delivery is created after production completes."*

4. C2 — Route-Based Product Classification

4.1 Overview

The fulfillment route's `route_category` replaces boolean flags as the single mechanism that determines whether a product requires manufacturing. Existing flags (`is_manufacturable`, `product_type`, `supply_method`) remain on the schema for backward compatibility and filtering but no longer drive system behavior.

4.2 Behavior Migration

Behavior	Before (ADD-030/031)	After (ADD-034)
BOM tab visibility	<code>product_type IN (FG, SF) OR supply_method = Manufacture OR is_manufacturable = true</code>	<code>route_category = 'MANUFACTURE'</code> on the product variant's fulfillment route
BOM mandatory check	<code>product_type IN (FG, SF)</code> → must have BOM	<code>route_category = 'MANUFACTURE'</code> → must have active BOM
SO confirmation → Production Order	Not implemented (manual PO creation)	Automatic when <code>route_category = 'MANUFACTURE'</code> (see C5)
Product Classification tab	Shows Product Type, Supply Method, Is Manufacturable toggles	Same fields remain as filter/segregation aids; behavior driven by route
Filter/segregation	<code>WHERE is_manufacturable = 1</code> or <code>WHERE product_type = 'FinishedGood'</code>	<code>WHERE fr.route_category = 'MANUFACTURE'</code> (JOIN on route). Legacy filters continue to work for backward compatibility.

4.3 BOM Tab Visibility Rule

The BOM tab/section on the Product form (Addendum 31 C4) is now driven by the variant's route:

csharp

```
// Before (ADD-031 C4 §6.2):
bool showBom = productType.In(FinishedGood, SemiFinished)
    || supplyMethod == Manufacture
    || isManufacturable;

// After (ADD-034 C2):
bool showBom = variant.FulfillmentRoute?.RouteCategory == "MANUFACTURE";
```

When the BOM section is visible (route_category = MANUFACTURE):

- At least one active BOM is required before the product can be used on a Sale Order
- Validation: "This product's fulfillment route requires manufacturing. At least one active Bill of Material is required."

When the BOM section is hidden (route_category ≠ MANUFACTURE):

- No BOM validation is applied
- Existing BOMs (if any) remain in the database but the tab is hidden
- If the user later changes the route to a manufacturing route, existing BOMs become visible again

4.4 Product Classification Tab — Updated Layout

The "Manufacturing & Supply" section (Addendum 31 C2, §4.2) now includes the route reference:

```
— Manufacturing & Supply —

Fulfillment Route: [Manufacture → Pick, Pack & Ship ▼]
Route Category:    MANUFACTURE (read-only badge)

Product Type:      [FinishedGood ▼]    ← retained for filtering
Supply Method:     [Manufacture ▼]     ← retained for filtering
☐ Is Manufacturable          ← auto-computed from route
☐ Is BOM Input      ☐ Is Saleable      ← retained
☐ Is Purchasable   ☐ Is Stockable     ← retained
```

4.5 Auto-Computed Flags

When a user assigns a fulfillment route to a product variant, the following fields are auto-updated:

Flag	Computation	Editable
is_manufacturable	<code>route_category == 'MANUFACTURE'</code> → TRUE, else FALSE	Read-only (syncd from route)
supply_method	<code>route_category == 'MANUFACTURE'</code> → 'Manufacture', else keep current value	Editable (user can override)

 **NOTE:** Auto-computation fires on route assignment change only. It does not retroactively update products when a route's category changes. Route category changes on an in-use route are blocked (see BR-C1-07).

4.6 Filter/Segregation Queries

For list views and reports, both old and new filters work:

```
sql

-- New standard (preferred):
SELECT pv.*, fr.route_category
FROM inventory.ProductVariants pv
JOIN logistics.FulfillmentRoutes fr ON pv.fulfillment_route_id = fr.fulfillment_route_id
WHERE fr.route_category = 'MANUFACTURE';

-- Legacy (backward compatible):
SELECT * FROM inventory.ProductVariants WHERE is_manufacturable = 1;
-- Returns same results because is_manufacturable is synced from route
```

5. C3 — Lead Time Management on Products

5.1 Overview

Add a "Lead Times" tab to the Product Variant detail screen. This tab presents a step-by-step breakdown of every time component contributing to the total lead time, configurable per variant with organization-level defaults as fallback.

5.2 Lead Time Components

Component	Column Name	Applies When	Description
Supplier Lead Time	supplier_lead_time_days	Route involves procurement	Time from PO submission to goods receipt. Sourced from preferred supplier record if not overridden here.
Manufacturing Lead Time	manufacturing_lead_time_days	route_category = MANUFACTURE	Time to produce one batch. Derived from BOM operations (C4) or set manually.
Manufacturing Buffer	manufacturing_buffer_days	route_category = MANUFACTURE	Safety buffer for production delays (machine breakdown, rework).
Quality Inspection Time	quality_inspection_days	QC enabled for product	Time for incoming or post-production inspection.
Internal Transfer Time	internal_transfer_days	Multi-warehouse setups	Time to move goods between internal locations.
Pick & Pack Time	pick_pack_days	All routes	Warehouse processing time for picking and packing.
Shipping Lead Time	shipping_lead_time_days	Route includes SHIP step	Transit time to customer.
Sales Safety Buffer	sales_buffer_days	All routes	Extra buffer added to promised date to account for unknowns.

5.3 Schema Change — ProductVariants

Modify: `inventory.ProductVariants`

ProductVariants — New Columns (ADD)

Column	Type	Nullable	Default	Description
supplier_lead_time_days	INT	NULL		Override of supplier-level lead time. NULL = use supplier record.
manufacturing_lead_time_days	INT	NULL		Calculated from BOM operations or manual entry.
manufacturing_buffer_days	INT	NULL		Default 0. Safety buffer for production delays.
quality_inspection_days	INT	NULL		Default 0. QC processing time.
internal_transfer_days	INT	NULL		Default 0. Internal movement time.
pick_pack_days	INT	NULL		NULL = use org default.
shipping_lead_time_days	INT	NULL		NULL = use org default.
sales_buffer_days	INT	NULL		NULL = use org default.



NOTE: NULL on any field means "use the organization default from lookups.LeadTimeDefaults." If the org default is also NULL, the component defaults to 0 days.

5.4 New Table — LeadTimeDefaults

New Table: `lookups.LeadTimeDefaults`

Column	Type	Nullable	Default	Description
lead_time_default_id	INT IDENTITY	NOT NULL	PK	
org_id	INT	NOT NULL	FK → tenants.Organizations	One row per org
pick_pack_days	INT	NOT NULL	1	Default warehouse processing time
shipping_lead_time_days	INT	NOT NULL	3	Default transit time
sales_buffer_days	INT	NOT NULL	1	Default safety buffer
manufacturing_buffer_days	INT	NOT NULL	0	Default production buffer
quality_inspection_days	INT	NOT NULL	0	Default QC time
internal_transfer_days	INT	NOT NULL	0	Default internal movement time
created_at	DATETIME2	NOT NULL	SYSUTCDATETIME()	
updated_at	DATETIME2	NOT NULL	SYSUTCDATETIME()	

Index:

Index	Columns	Type
UQ_LeadTimeDefaults_Org	org_id	Unique

5.5 Effective Lead Time Resolution

csharp

```
// Pseudocode – LeadTimeResolver
public int ResolveComponent(ProductVariant variant, string component, LeadTimeDefaults orgDefaults)
{
    // Priority 1: Variant-level override
    int? variantValue = component switch
    {
        "pick_pack" => variant.PickPackDays,
        "shipping" => variant.ShippingLeadTimeDays,
        "sales_buffer" => variant.SalesBufferDays,
        "mfg_buffer" => variant.ManufacturingBufferDays,
        "qc" => variant.QualityInspectionDays,
        "transfer" => variant.InternalTransferDays,
        _ => null
    };
    if (variantValue.HasValue) return variantValue.Value;

    // Priority 2: Org default
    int orgValue = component switch
    {
        "pick_pack" => orgDefaults.PickPackDays,
        "shipping" => orgDefaults.ShippingLeadTimeDays,
        "sales_buffer" => orgDefaults.SalesBufferDays,
        "mfg_buffer" => orgDefaults.ManufacturingBufferDays,
        "qc" => orgDefaults.QualityInspectionDays,
        "transfer" => orgDefaults.InternalTransferDays,
        _ => 0
    };

    return orgValue; // Org defaults have their own non-null defaults (0 minimum)
}
```

5.6 Component Visibility by Route

Not all components are relevant for every route. The Lead Times tab hides inapplicable components:

Component	STOCK routes	MANUFACTURE routes	Visibility Rule
Supplier Lead Time	✓ Shown	✓ Shown (for raw materials)	Always shown
Manufacturing Lead Time	✗ Hidden	✓ Shown	route_category = 'MANUFACTURE'
Manufacturing Buffer	✗ Hidden	✓ Shown	route_category = 'MANUFACTURE'
Quality Inspection	✓ Shown (optional)	✓ Shown (optional)	Always shown, default 0
Internal Transfer	✓ Shown (optional)	✓ Shown (optional)	Always shown, default 0
Pick & Pack Time	✓ Shown	✓ Shown	Always shown
Shipping Lead Time	✓ Shown if SHIP step	✓ Shown if SHIP step	Route includes SHIP step
Sales Safety Buffer	✓ Shown	✓ Shown	Always shown

6. C4 — Per-Line Lead Time Calculation with BOM Awareness

6.1 Overview

A dedicated (`ILeadTimeCalculator`) service computes the total lead time for a product variant on demand. The calculation traverses the BOM tree for manufacturing products, accounts for sub-component availability, and returns a component-by-component breakdown. A "Calculate" button is available on each line item at the Inquiry, Quotation, and Sale Order levels.

6.2 Service Interface

```
csharp
```

```
public interface ILeadTimeCalculator
{
    Task<LeadTimeResult> CalculateAsync(LeadTimeRequest request);
}

public class LeadTimeRequest
{
    public long VariantId { get; set; }
    public decimal Quantity { get; set; }
    public DateTimeOffset? RequestedDate { get; set; } // optional: backward calculation
    public int OrgId { get; set; }
}

public class LeadTimeResult
{
    public int TotalLeadTimeDays { get; set; }
    public DateTimeOffset EarliestDeliveryDate { get; set; }
    public List<LeadTimeComponent> Components { get; set; }
}

public class LeadTimeComponent
{
    public string Name { get; set; } // e.g., "Manufacturing Lead Time"
    public int Days { get; set; }
    public string Source { get; set; } // "BOM Calculation", "Manual", "Org Default",
}
```

6.3 Calculation Algorithm

csharp

```

// Pseudocode – LeadTimeCalculator.CalculateAsync()
public async Task<LeadTimeResult> CalculateAsync(LeadTimeRequest request)
{
    var variant = await _variantRepo.GetWithRouteAsync(request.VariantId);
    var orgDefaults = await _leadTimeDefaultsRepo.GetByOrgAsync(request.OrgId);
    var route = variant.FulfillmentRoute;
    var components = new List<LeadTimeComponent>();

    // 1. Manufacturing lead time (if MANUFACTURE route)
    if (route?.RouteCategory == "MANUFACTURE")
    {
        int mfgDays = variant.ManufacturingLeadTimeDays
            ?? await CalculateManufacturingLeadTime(request.VariantId, request.Quantity);

        components.Add(new("Manufacturing", mfgDays,
            variant.ManufacturingLeadTimeDays.HasValue ? "Manual" : "BOM Calculation"));

        int mfgBuffer = ResolveComponent(variant, "mfg_buffer", orgDefaults);
        if (mfgBuffer > 0)
            components.Add(new("Manufacturing Buffer", mfgBuffer, "Variant/Org Default"));
    }

    // 2. Supplier lead time (for procured products)
    if (route?.RouteCategory != "MANUFACTURE")
    {
        int supplierDays = variant.SupplierLeadTimeDays
            ?? await GetPreferredSupplierLeadTime(request.VariantId);
        if (supplierDays > 0)
            components.Add(new("Supplier Lead Time", supplierDays,
                variant.SupplierLeadTimeDays.HasValue ? "Manual" : "Supplier Record"));
    }

    // 3. QC time
    int qcDays = ResolveComponent(variant, "qc", orgDefaults);
    if (qcDays > 0) components.Add(new("Quality Inspection", qcDays, "Variant/Org Default"));

    // 4. Internal transfer
    int transferDays = ResolveComponent(variant, "transfer", orgDefaults);
    if (transferDays > 0) components.Add(new("Internal Transfer", transferDays, "Variant/Org Default"));

    // 5. Pick & pack

```

```

int pickPackDays = ResolveComponent(variant, "pick_pack", orgDefaults);
components.Add(new("Pick & Pack", pickPackDays, "Variant/Org Default"));

// 6. Shipping (if route has SHIP step)
if (route?.RequiresShipping == true)
{
    int shipDays = ResolveComponent(variant, "shipping", orgDefaults);
    components.Add(new("Shipping", shipDays, "Variant/Org Default"));
}

// 7. Sales buffer
int bufferDays = ResolveComponent(variant, "sales_buffer", orgDefaults);
if (bufferDays > 0) components.Add(new("Sales Safety Buffer", bufferDays, "Variant/Org Default"));

// Sum
int totalDays = components.Sum(c => c.Days);
var earliestDate = DateTimeOffset.UtcNow.AddDays(totalDays);

return new LeadTimeResult
{
    TotalLeadTimeDays = totalDays,
    EarliestDeliveryDate = earliestDate,
    Components = components
};
}

```

6.4 BOM-Aware Manufacturing Lead Time Calculation

csharp

```

// Pseudocode – CalculateManufacturingLeadTime (recursive)
private async Task<int> CalculateManufacturingLeadTime(
    long variantId, decimal quantity, HashSet<long>? visited = null)
{
    visited ??= new HashSet<long>();

    // Cycle detection
    if (!visited.Add(variantId))
    {
        _logger.LogWarning("Circular BOM detected for variant {VariantId}", variantId);
        return 0;
    }

    var bom = await _bomRepo.GetActiveBomAsync(variantId);
    if (bom == null) return 0;

    // Step 1: This level's production time (from BOM operations)
    int thisLevelDays = 0;
    if (bom.Operations?.Any() == true)
    {
        decimal totalMinutes = 0;
        foreach (var op in bom.Operations.OrderBy(o => o.SequenceOrder))
        {
            decimal opMinutes = op.SetupTimeMinutes
                + (op.CycleTimeMinutes * Math.Ceiling(quantity / op.BatchSize));
            totalMinutes += opMinutes;
        }
        // Convert minutes to working days (8 hours/day)
        thisLevelDays = (int)Math.Ceiling(totalMinutes / 480m);
    }
    else
    {
        // No operations defined – use variant's manual manufacturing_lead_time_days
        var variant = await _variantRepo.GetByIdAsync(variantId);
        thisLevelDays = variant.ManufacturingLeadTimeDays ?? 1; // fallback to 1 day
    }

    // Step 2: Sub-component lead times (parallel – take the max)
    int maxComponentLead = 0;
    foreach (var component in bom.Lines)
    {

```

```

decimal requiredQty = component.QuantityPer * quantity;
decimal available = await _stockService.GetAvailableQtyAsync(
    component.VariantId, /* all warehouses */);
decimal shortfall = requiredQty - available;

if (shortfall > 0)
{
    var compVariant = await _variantRepo.GetWithRouteAsync(component.VariantId);
    int compLead;

    if (compVariant.FulfillmentRoute?.RouteCategory == "MANUFACTURE")
    {
        // Recursive: sub-assembly needs production
        compLead = await CalculateManufacturingLeadTime(
            component.VariantId, shortfall, visited);
    }
    else
    {
        // Procured component: use supplier lead time
        compLead = compVariant.SupplierLeadTimeDays
            ?? await GetPreferredSupplierLeadTime(component.VariantId);
    }

    maxComponentLead = Math.Max(maxComponentLead, compLead);
}
// If no shortfall (stock exists), component lead = 0
}

// Step 3: Total = parallel component wait + sequential production
return maxComponentLead + thisLevelDays;
}

```

Key design decisions:

- Sub-components are treated as **parallel** — all are ordered/produced at once; wait for the longest one.
- This level's operations run **after** all components arrive — sequential addition.
- Stock-aware — if components are in stock, their lead time is 0.
- Cycle detection — visited set prevents infinite recursion in circular BOMs.
- Cache results for 5 minutes keyed by (variant_id, quantity_bucket) for performance.

6.5 Schema Changes — Line Item Tables

Add lead time tracking columns to Inquiry, Quotation, and Sale Order line tables:

Modify: `demand.InquiryLines`, `demand.QuotationLines`, `demand.SaleOrderLines`

New Columns (ADD to each table)

Column	Type	Nullable	Description
calculated_lead_time_days	INT	NULL	Last calculated total lead time
calculated_delivery_date	DATE	NULL	Computed: today + lead time
manual_delivery_date	DATE	NULL	User override. NULL = use calculated.
lead_time_calculated_at	DATETIME2	NULL	When the last calculation ran

Computed Column (ADD to each table):

```
sql
effective_delivery_date AS COALESCE(manual_delivery_date, calculated_delivery_date) PERSI
```

6.6 Line-Item Calculate Button

Each line item at Inquiry, Quotation, and Sale Order levels gets a  **Calculate** button:

1. User selects a product variant on a line item.
2. User clicks the  button.
3. System calls `POST /api/lead-time/calculate` with `{ variant_id, quantity }`.
4. A popover expands below the line showing each component with its value and source.
5. The `calculated_delivery_date` auto-fills on the line.
6. User can override by editing the delivery date directly (stored as `manual_delivery_date`).

6.7 Manual Override Indicator

When a user manually changes the delivery date, the line shows:

State	Indicator	Tooltip
Calculated	 Calculated (muted)	"Lead time: 12 days — calculated on Oct 3, 2026"
Manual Override	 Manual (amber)	"Manual date — calculated was Oct 15, 2026"
Not Calculated	 Not calculated (grey)	"Click  to calculate lead time"

7. C5 — SO Confirmation: Split by Route Category

7.1 Overview

Modify the Sale Order confirmation logic (Addendum 33 C4, `SaleOrderDeliveryCreator`) to split behavior by route category. Lines with `STOCK` routes create Delivery Orders immediately (existing behavior). Lines with `MANUFACTURE` routes create Production Orders instead — delivery is deferred to production completion (C6).

7.2 Modified SO Confirmation Flow

SO Confirmation (DRAFT → CONFIRMED)

- | — Validation (Addendum 33 C3): all lines must have effective route ✓
- | — FOR EACH SO line, grouped by route_category:
 - | | — route_category = 'STOCK'
 - | | | — SaleOrderDeliveryCreator.CreateDeliveriesFromSaleOrder()
 - | | | → Group stock lines by route
 - | | | → Create Delivery Order per route group
 - | | | → Delivery status = DRAFT
 - | | | → (unchanged from Addendum 33 C4)
 - | | — route_category = 'MANUFACTURE'
 - | | | — SaleOrderProductionCreator.CreateProductionFromSaleOrder()
 - | | | → Create Production Order per manufacturing line
 - | | | → PO.sale_order_id = SO.sale_order_id
 - | | | → PO.sale_order_line_id = line.sale_order_line_id
 - | | | → PO.fulfillment_route_id = line's effective route
 - | | | → PO.status = DRAFT
 - | | | → PO.planned_qty = line.quantity
 - | | | → PO.variant_id = line.variant_id
 - | | | → PO.production_warehouse_id = variant.default_production_warehouse_id
 - | | | → Trigger BOM explosion (existing flow)
- | — Update SO status → CONFIRMED

7.3 New Service — SaleOrderProductionCreator

csharp

```
// Pseudocode – SaleOrderProductionCreator
public class SaleOrderProductionCreator
{
    public async Task<List<ProductionOrder>> CreateProductionFromSaleOrder(
        SaleOrder order, IEnumerable<SaleOrderLine> manufacturingLines)
    {
        var productionOrders = new List<ProductionOrder>();

        foreach (var line in manufacturingLines)
        {
            var variant = await _variantRepo.GetWithRouteAsync(line.VariantId);
            var effectiveRouteId = _routeResolver.ResolveEffectiveRouteId(line);

            var po = new ProductionOrder
            {
                OrgId = order.OrgId,
                ProductId = line.ProductId,
                VariantId = line.VariantId,
                PlannedQty = line.Quantity,
                Status = "DRAFT",
                ProductionWarehouseId = variant.DefaultProductionWarehouseId,
                PlannedStartDate = CalculatePlannedStart(line),
                PlannedEndDate = CalculatePlannedEnd(line, variant),
                SaleOrderId = order.SaleOrderId,
                SaleOrderLineId = line.SaleOrderLineId,
                FulfillmentRouteId = effectiveRouteId,
                SourceDemandType = "SALES_ORDER",
                SourceDemandId = order.SaleOrderId,
                CreatedBy = "SYSTEM"
            };

            productionOrders.Add(po);
        }

        await _productionOrderRepo.CreateBatchAsync(productionOrders);

        // Trigger BOM explosion for each PO
        foreach (var po in productionOrders)
        {
            await _bomExplosionService.ExplodeAsync(po.ProductionOrderId);
        }
    }
}
```

```

        // Notify production planning
        await _notificationService.NotifyAsync(
            NotificationType.ProductionOrderCreatedFromSO,
            order.OrgId,
            new { SaleOrderNumber = order.OrderNumber, Count = productionOrders.Count });

        return productionOrders;
    }

    private DateTimeOffset CalculatePlannedStart(SaleOrderLine line)
    {
        // If delivery date is known, backward-schedule from it
        if (line.EffectiveDeliveryDate.HasValue)
        {
            // Subtract shipping + pick/pack time to get production start
            var variant = _variantRepo.GetById(line.VariantId);
            int postProductionDays = (variant.PickPackDays ?? 1)
                + (variant.ShippingLeadTimeDays ?? 3)
                + (variant.SalesBufferDays ?? 1);
            int mfgDays = variant.ManufacturingLeadTimeDays ?? 5;
            return line.EffectiveDeliveryDate.Value.AddDays(-(postProductionDays + mfgDays));
        }
        // Default: start tomorrow
        return DateTimeOffset.UtcNow.AddDays(1);
    }
}

```

7.4 Modified SaleOrderDeliveryCreator

The existing `SaleOrderDeliveryCreator` (Addendum 33 C4) is modified to EXCLUDE manufacturing lines:

csharp

```

// Modified – SaleOrderDeliveryCreator.CreateDeliveriesFromSaleOrder()
public async Task<List<DeliveryOrder>> CreateDeliveriesFromSaleOrder(SaleOrder order)
{
    // CHANGED: Only process non-manufacturing lines
    var stockLines = order.Lines
        .Where(l =>
        {
            var routeId = _routeResolver.ResolveEffectiveRouteId(l);
            var route = _routeRepo.GetById(routeId!.Value);
            return route.RouteCategory != "MANUFACTURE";
        })
        .ToList();

    if (!stockLines.Any())
        return new List<DeliveryOrder>(); // All lines are manufacturing – no immediate d

    // Group stock lines by route (existing logic, unchanged)
    var linesByRoute = stockLines
        .GroupBy(l => _routeResolver.ResolveEffectiveRouteId(l)!.Value)
        .ToList();

    // ... rest unchanged from Addendum 33 C4 ...
}

```

7.5 Orchestrating Confirmation

csharp

```
// SaleOrderService.ConfirmAsync() – orchestration
public async Task ConfirmAsync(long saleOrderId)
{
    var order = await _repo.GetWithLinesAsync(saleOrderId);

    // Step 1: Validate routes (unchanged from Addendum 33 C3)
    var validation = _confirmationValidator.ValidateForConfirmation(order);
    if (!validation.IsValid) throw new BusinessException(validation.Errors);

    // Step 2: Split lines by route category
    var linesByCategory = order.Lines
        .GroupBy(l =>
        {
            var routeId = _routeResolver.ResolveEffectiveRouteId(l);
            var route = _routeRepo.GetById(routeId!.Value);
            return route.RouteCategory;
        });

    var stockLines = linesByCategory
        .Where(g => g.Key != "MANUFACTURE")
        .SelectMany(g => g)
        .ToList();

    var mfgLines = linesByCategory
        .Where(g => g.Key == "MANUFACTURE")
        .SelectMany(g => g)
        .ToList();

    // Step 3: Create deliveries for stock lines
    if (stockLines.Any())
        await _deliveryCreator.CreateDeliveriesFromSaleOrder(order, stockLines);

    // Step 4: Create production orders for manufacturing lines
    if (mfgLines.Any())
        await _productionCreator.CreateProductionFromSaleOrder(order, mfgLines);

    // Step 5: Update SO status
    order.Status = "CONFIRMED";
    order.ConfirmedAt = DateTimeOffset.UtcNow;
    await _repo.UpdateAsync(order);
}
```

```
// Step 6: Publish event
await _mediator.Publish(new SaleOrderConfirmedEvent(order));
}
```

7.6 Mixed Orders

A Sale Order can have both stock and manufacturing lines. Both are processed at confirmation:

```
Sale Order SO-2001 confirmed
├─ Line 1: Steel Pipes (Route: PICK_AND_SHIP, category=STOCK)
│   └─ Delivery DO-001 created immediately (Addendum 33 flow)
├─ Line 2: Custom Gear Assembly (Route: MFG_PICK_PACK_SHIP, category=MANUFACTURE)
│   └─ Production Order PROD-085 created (delivery deferred)
└─ Line 3: Copper Fittings (Route: PICK_PACK_SHIP, category=STOCK)
    └─ Delivery DO-001 includes this line (same route as Line 1? No – different route,
        so DO-002)

Result at confirmation:
• DO-001: Steel Pipes (PICK_AND_SHIP) – immediate delivery
• DO-002: Copper Fittings (PICK_PACK_SHIP) – immediate delivery
• PROD-085: Custom Gear Assembly – production first, delivery after FGR
```

7.7 SO Fulfillment Tab — Updated View

The Sale Order Fulfillment tab (Addendum 33 §10.4) now shows both delivery orders AND production orders:

```
Fulfillment Overview
├─ Deliveries (2)
│   ├── DO-001: Steel Pipes – • Picking
│   └─ DO-002: Copper Fittings – • Draft
└─ Production Orders (1)
    └─ PROD-085: Custom Gear Assembly – • MaterialPending
        └─ Delivery: pending production completion
```

8. C6 — Production Completion → Delivery Auto-Creation

8.1 Overview

When a Production Order linked to a Sale Order completes (FGR confirmed, status → Completed), the system automatically creates a Delivery Order for the SO line using the route

stored on the Production Order. Standalone Production Orders (no SO link) do not trigger delivery — finished goods go to inventory.

8.2 Trigger Point

Event: `FGRConfirmedEvent` (existing MediatR event, fired when Finished Goods Receipt is confirmed)

Extended handler:

csharp

```
// Extended – FGRConfirmedEventHandler
public class FGRConfirmedEventHandler : INotificationHandler<FGRConfirmedEvent>
{
    public async Task Handle(FGRConfirmedEvent notification, CancellationToken ct)
    {
        var po = await _productionOrderRepo.GetByIdAsync(notification.ProductionOrderId);

        // EXISTING: Automatic allocation to SO demands
        await _allocationEngine.AllocateForFGR(po);

        // NEW: Auto-create delivery if PO is linked to a Sale Order
        if (po.SaleOrderId.HasValue && po.FulfillmentRouteId.HasValue)
        {
            await _productionDeliveryCreator.CreateDeliveryFromProduction(po);
        }
        // ELSE: Standalone production – FG goes to inventory, no auto-delivery
    }
}
```

8.3 Delivery Creation from Production

csharp

```

// Pseudocode – ProductionDeliveryCreator
public class ProductionDeliveryCreator
{
    public async Task<DeliveryOrder> CreateDeliveryFromProduction(ProductionOrder po)
    {
        var saleOrder = await _saleOrderRepo.GetByIdAsync(po.SaleOrderId!.Value);
        var soLine = await _soLineRepo.GetByIdAsync(po.SaleOrderLineId!.Value);
        var route = await _routeRepo.GetWithStepsAsync(po.FulfillmentRouteId!.Value);

        // Delivery quantity = actual produced quantity
        // (may differ from planned if production had yield loss)
        decimal deliveryQty = po.ActualQty;

        var delivery = new DeliveryOrder
        {
            OrgId = po.OrgId,
            FromSourceType = "SALE_ORDER",
            SaleOrderId = po.SaleOrderId.Value,
            FulfillmentRouteId = route.FulfillmentRouteId,
            WarehouseId = po.ProductionWarehouseId,
            Status = "DRAFT",
            ExpectedDate = soLine.EffectiveDeliveryDate,
            PartnerId = saleOrder.PartnerId,
            DeliveryAddressId = saleOrder.DeliveryAddressId,
            Lines = new List<DeliveryOrderLine>
            {
                new DeliveryOrderLine
                {
                    VariantId = po.VariantId,
                    Quantity = deliveryQty,
                    SaleOrderLineId = po.SaleOrderLineId.Value,
                    OrgId = po.OrgId
                }
            }
        };

        await _deliveryRepo.CreateAsync(delivery);

        // Link back: update PO with delivery reference
        po.DeliveryOrderId = delivery.DeliveryOrderId;
        await _productionOrderRepo.UpdateAsync(po);
    }
}

```



```
// Update SO Fulfillment tab
await _saleOrderService.RecalculateFulfillmentStatusAsync(po.SaleOrderId.Value);

// Notify warehouse team
await _notificationService.NotifyAsync(
    NotificationType.DeliveryCreatedFromProduction,
    po.OrgId,
    new
    {
        DeliveryNumber = delivery.DeliveryNumber,
        ProductionOrderNumber = po.ProductionOrderNumber,
        SaleOrderNumber = saleOrder.OrderNumber,
        RouteName = route.Name
    });

return delivery;
}
}
```

8.4 End-to-End Flow

```
Customer places order
  ↓
Sale Order SO-2001 created (DRAFT)
  ↓
Line 2: Custom Gear Assembly, Route: MFG_PICK_PACK_SHIP (MANUFACTURE)
  ↓
SO Confirmed → Production Order PROD-085 created
  ├── sale_order_id = SO-2001
  ├── sale_order_line_id = line 2
  ├── fulfillment_route_id = MFG_PICK_PACK_SHIP
  └── status = DRAFT
  ↓
BOM Explosion → PMRs created → Material procurement
  ↓
Materials received (GRN) → Allocation → Materials reserved for PROD-085
  ↓
Production starts → Manufacturing → Quality Inspection → FGR Confirmed
  ↓
FGRConfirmedEvent fires:
  ├── Existing: Allocate FG stock
  └── NEW: CreateDeliveryFromProduction()
    ↓
Delivery Order DO-20261020-003 created
  ├── from_source_type = SALE_ORDER
  ├── sale_order_id = SO-2001
  ├── fulfillment_route_id = MFG_PICK_PACK_SHIP
  ├── Line: Custom Gear Assembly × 95 (actual produced qty)
  └── Steps: PICK → PACK → GOODS_ISSUE → SHIP
    ↓
Warehouse releases delivery → Pick → Pack → Goods Issue → Ship → Delivered
```

8.5 Standalone Production (No SO Link)

Production Order PROD-100 created directly (no SO)

- |— sale_order_id = NULL
- |— sale_order_line_id = NULL
- |— fulfillment_route_id = NULL
- |— status = DRAFT

↓

Manufacturing → FGR Confirmed

↓

FGRConfirmedEvent fires:

- |— Allocation: FG stock added to inventory
- |— Delivery check: sale_order_id IS NULL → NO delivery created

↓

Finished goods sit in inventory (Make-to-Stock)

↓

Later: Customer orders the product → SO-2050 created

- |— Product has route MFG_PICK_PACK_SHIP (category = MANUFACTURE)
- |— Salesperson overrides route on SO line to PICK_PACK_SHIP (category = STOCK)
| because stock is already available from prior production
- |— SO Confirmed → Delivery created immediately (stock route)

↓

OR: Salesperson keeps manufacturing route → new Production Order created

 **NOTE:** For MTS scenarios, the salesperson's route override on the SO line is the escape hatch. If stock exists from a prior production run, the salesperson selects a stock route (e.g., PICK_PACK_SHIP) to fulfill from inventory without triggering another production run.

8.6 Partial Production

When a Production Order produces less than planned (yield loss, partial completion):

Scenario	Delivery Qty	Action
Planned: 100, Actual: 100	Delivery for 100	Normal flow
Planned: 100, Actual: 95	Delivery for 95	Shortfall of 5 noted on SO line. User decides: adjust SO qty, create another PO, or leave as partial fulfillment.
Planned: 100, Actual: 0	No delivery created	PO completed with zero yield — error scenario. Notification sent to production manager.

Shortfall handling:

```
csharp

if (po.ActualQty < soLine.Quantity)
{
    // Create delivery for actual qty
    // Flag the SO line as partially fulfilled
    soLine.PartialFulfillmentNotes =
        $"Production Order {po.ProductionOrderNumber} produced {po.ActualQty} " +
        $"of {soLine.Quantity}. Shortfall: {soLine.Quantity - po.ActualQty}";

    await _notificationService.NotifyAsync(
        NotificationType.ProductionShortfall,
        po.OrgId,
        new { ProductionOrderNumber = po.ProductionOrderNumber,
            Planned = po.PlannedQty, Actual = po.ActualQty,
            SaleOrderNumber = saleOrder.OrderNumber });
}
```

8.7 Production Order — New Column

Modify: `material.ProductionOrders`

ProductionOrders — New Column (ADD)

Column	Type	Nullable	Description
fulfillment_route_id	INT	NULL	FK → logistics.FulfillmentRoutes. Resolved from SO line's effective route at PO creation. Used to create delivery after FGR. NULL for standalone POs.
delivery_order_id	BIGINT	NULL	FK → logistics.delivery_orders. Populated when delivery is auto-created from FGR. NULL before production completes and for standalone POs.

9. Database Migrations

Four migrations are required for this addendum.

9.1 Migration Order

#	Migration Name	Schema	Type	Description
M1	AddRouteCategoryToFulfillmentRoutes	logistics	ALTER TABLE + INSERT	Add route_category column, insert manufacturing seed routes
M2	CreateLeadTimeDefaults	lookups	CREATE TABLE + ALTER TABLE	New LeadTimeDefaults table + lead time columns on ProductVariants
M3	AddLeadTimeColumnsToLineItems	demand	ALTER TABLE	Add lead time columns to InquiryLines, QuotationLines, SaleOrderLines
M4	AddRouteAndDeliveryToProductionOrders	material	ALTER TABLE	Add fulfillment_route_id and delivery_order_id to ProductionOrders

9.2 Migration Details

M1 — AddRouteCategoryToFulfillmentRoutes

sql

```

-- Add route_category to existing FulfillmentRoutes table
ALTER TABLE logistics.FulfillmentRoutes
ADD route_category NVARCHAR(20) NOT NULL
    CONSTRAINT DF_FulfillmentRoutes_Category DEFAULT 'STOCK'
    CONSTRAINT CK_FulfillmentRoutes_Category
    CHECK (route_category IN ('STOCK', 'MANUFACTURE', 'BUY', 'DROPSHIP'));

-- Backfill existing routes: all current routes are STOCK
-- (they were created in Addendum 33 without manufacturing awareness)
UPDATE logistics.FulfillmentRoutes SET route_category = 'STOCK';

-- Insert new manufacturing seed routes per existing org
-- (executed via stored procedure in tenant provisioning;
-- sample for one org shown here)

DECLARE @orgId INT;
DECLARE org_cursor CURSOR FOR
    SELECT DISTINCT org_id FROM logistics.FulfillmentRoutes;

OPEN org_cursor;
FETCH NEXT FROM org_cursor INTO @orgId;

WHILE @@FETCH_STATUS = 0
BEGIN
    -- MFG_PICK_SHIP
    INSERT INTO logistics.FulfillmentRoutes
        (org_id, code, name, description, route_category, is_default, is_active,
         is_system, requires_packing, requires_shipping, display_order)
    VALUES
        (@orgId, 'MFG_PICK_SHIP', 'Manufacture → Pick & Ship',
         'Product manufactured then picked and shipped without packing.',
         'MANUFACTURE', 0, 1, 1, 0, 1, 40);

    DECLARE @mfgPickShipId INT = SCOPE_IDENTITY();

    INSERT INTO logistics.FulfillmentRouteSteps
        (fulfillment_route_id, org_id, step_code, step_order, is_mandatory)
    VALUES
        (@mfgPickShipId, @orgId, 'PICK', 1, 1),
        (@mfgPickShipId, @orgId, 'GOODS_ISSUE', 2, 1),
        (@mfgPickShipId, @orgId, 'SHIP', 3, 1);

```

```

-- MFG_PICK_PACK_SHIP
INSERT INTO logistics.FulfillmentRoutes
    (org_id, code, name, description, route_category, is_default, is_active,
     is_system, requires_packing, requires_shipping, display_order)
VALUES
    (@orgId, 'MFG_PICK_PACK_SHIP', 'Manufacture → Pick, Pack & Ship',
     'Product manufactured then picked, packed and shipped.',
     'MANUFACTURE', 0, 1, 1, 1, 1, 50);

DECLARE @mfgPickPackShipId INT = SCOPE_IDENTITY();

INSERT INTO logistics.FulfillmentRouteSteps
    (fulfillment_route_id, org_id, step_code, step_order, is_mandatory)
VALUES
    (@mfgPickPackShipId, @orgId, 'PICK', 1, 1),
    (@mfgPickPackShipId, @orgId, 'PACK', 2, 1),
    (@mfgPickPackShipId, @orgId, 'GOODS_ISSUE', 3, 1),
    (@mfgPickPackShipId, @orgId, 'SHIP', 4, 1);

    FETCH NEXT FROM org_cursor INTO @orgId;
END;

CLOSE org_cursor;
DEALLOCATE org_cursor;

```

M2 — CreateLeadTimeDefaults

```
sql
```

```

-- New table: org-level lead time defaults
CREATE TABLE lookups.LeadTimeDefaults (
    lead_time_default_id INT IDENTITY(1,1) NOT NULL
        CONSTRAINT PK_LeadTimeDefaults PRIMARY KEY,
    org_id INT NOT NULL
        CONSTRAINT FK_LeadTimeDefaults_Org FOREIGN KEY REFERENCES tenants.Organizations(o
    pick_pack_days INT NOT NULL CONSTRAINT DF_LTD_PickPack DEFAULT 1,
    shipping_lead_time_days INT NOT NULL CONSTRAINT DF_LTD_Shipping DEFAULT 3,
    sales_buffer_days INT NOT NULL CONSTRAINT DF_LTD_SalesBuffer DEFAULT 1,
    manufacturing_buffer_days INT NOT NULL CONSTRAINT DF_LTD_MfgBuffer DEFAULT 0,
    quality_inspection_days INT NOT NULL CONSTRAINT DF_LTD_QC DEFAULT 0,
    internal_transfer_days INT NOT NULL CONSTRAINT DF_LTD_Transfer DEFAULT 0,
    created_at DATETIME2 NOT NULL CONSTRAINT DF_LTD_Created DEFAULT SYSUTCDATETIME(),
    updated_at DATETIME2 NOT NULL CONSTRAINT DF_LTD_Updated DEFAULT SYSUTCDATETIME(),

    CONSTRAINT UQ_LeadTimeDefaults_Org UNIQUE (org_id)
);

-- Seed one row per existing org with system defaults
INSERT INTO lookups.LeadTimeDefaults (org_id)
SELECT DISTINCT org_id FROM tenants.Organizations;

-- Add lead time columns to ProductVariants
ALTER TABLE inventory.ProductVariants ADD supplier_lead_time_days INT NULL;
ALTER TABLE inventory.ProductVariants ADD manufacturing_lead_time_days INT NULL;
ALTER TABLE inventory.ProductVariants ADD manufacturing_buffer_days INT NULL;
ALTER TABLE inventory.ProductVariants ADD quality_inspection_days INT NULL;
ALTER TABLE inventory.ProductVariants ADD internal_transfer_days INT NULL;
ALTER TABLE inventory.ProductVariants ADD pick_pack_days INT NULL;
ALTER TABLE inventory.ProductVariants ADD shipping_lead_time_days INT NULL;
ALTER TABLE inventory.ProductVariants ADD sales_buffer_days INT NULL;

-- Migrate existing lead_time_days value (Addendum 30/31)
-- to manufacturing_lead_time_days for manufacturing products
UPDATE pv
SET pv.manufacturing_lead_time_days = p.lead_time_days
FROM inventory.ProductVariants pv
JOIN lookups.Products p ON pv.product_id = p.product_id
WHERE p.lead_time_days IS NOT NULL
    AND p.lead_time_days > 0
    AND p.is_manufacturable = 1;

```


M3 — AddLeadTimeColumnsToLineItems

sql

```
-- InquiryLines
ALTER TABLE demand.InquiryLines ADD calculated_lead_time_days INT NULL;
ALTER TABLE demand.InquiryLines ADD calculated_delivery_date DATE NULL;
ALTER TABLE demand.InquiryLines ADD manual_delivery_date DATE NULL;
ALTER TABLE demand.InquiryLines ADD lead_time_calculated_at DATETIME2 NULL;
ALTER TABLE demand.InquiryLines ADD effective_delivery_date
    AS COALESCE(manual_delivery_date, calculated_delivery_date) PERSISTED;

-- QuotationLines
ALTER TABLE demand.QuotationLines ADD calculated_lead_time_days INT NULL;
ALTER TABLE demand.QuotationLines ADD calculated_delivery_date DATE NULL;
ALTER TABLE demand.QuotationLines ADD manual_delivery_date DATE NULL;
ALTER TABLE demand.QuotationLines ADD lead_time_calculated_at DATETIME2 NULL;
ALTER TABLE demand.QuotationLines ADD effective_delivery_date
    AS COALESCE(manual_delivery_date, calculated_delivery_date) PERSISTED;

-- SaleOrderLines
ALTER TABLE demand.SaleOrderLines ADD calculated_lead_time_days INT NULL;
ALTER TABLE demand.SaleOrderLines ADD calculated_delivery_date DATE NULL;
ALTER TABLE demand.SaleOrderLines ADD manual_delivery_date DATE NULL;
ALTER TABLE demand.SaleOrderLines ADD lead_time_calculated_at DATETIME2 NULL;
ALTER TABLE demand.SaleOrderLines ADD effective_delivery_date
    AS COALESCE(manual_delivery_date, calculated_delivery_date) PERSISTED;
```

M4 — AddRouteAndDeliveryToProductionOrders

sql

```
ALTER TABLE material.ProductionOrders
ADD fulfillment_route_id INT NULL
    CONSTRAINT FK_ProductionOrders_FulfillmentRoute
    FOREIGN KEY REFERENCES logistics.FulfillmentRoutes(fulfillment_route_id);

ALTER TABLE material.ProductionOrders
ADD delivery_order_id BIGINT NULL
    CONSTRAINT FK_ProductionOrders_DeliveryOrder
    FOREIGN KEY REFERENCES logistics.delivery_orders(delivery_order_id);

CREATE INDEX IX_ProductionOrders_FulfillmentRoute
    ON material.ProductionOrders(fulfillment_route_id)
    WHERE fulfillment_route_id IS NOT NULL;

CREATE INDEX IX_ProductionOrders_DeliveryOrder
    ON material.ProductionOrders(delivery_order_id)
    WHERE delivery_order_id IS NOT NULL;

-- Backfill: existing SO-linked Production Orders get their route from the SO line
UPDATE po
SET po.fulfillment_route_id = COALESCE(sol.fulfillment_route_id, pv.fulfillment_route_id)
FROM material.ProductionOrders po
JOIN demand.SaleOrderLines sol ON po.sale_order_line_id = sol.sale_order_line_id
JOIN inventory.ProductVariants pv ON po.variant_id = pv.variant_id
WHERE po.sale_order_id IS NOT NULL
    AND po.fulfillment_route_id IS NULL;
```

9.3 Rollback

Migration Rollback Steps

M1	Remove manufacturing seed routes (DELETE WHERE code IN ('MFG_PICK_SHIP','MFG_PICK_PACK_SHIP')); DROP COLUMN route_category from FulfillmentRoutes
M2	DROP TABLE lookups.LeadTimeDefaults; DROP COLUMNS supplier_lead_time_days, manufacturing_lead_time_days, etc. from ProductVariants
M3	DROP COLUMNS calculated_lead_time_days, calculated_delivery_date, manual_delivery_date, lead_time_calculated_at, effective_delivery_date from InquiryLines, QuotationLines, SaleOrderLines
M4	DROP COLUMNS fulfillment_route_id, delivery_order_id from ProductionOrders (after clearing FKs and indexes)

10. API Changes

10.1 New Endpoints

Endpoint	Method	Description	Permission
/api/lead-time/calculate	POST	Calculate lead time for a variant with quantity. Returns component breakdown + earliest delivery date.	authenticated
/api/lead-time/calculate-manufacturing	POST	Calculate manufacturing lead time only (BOM traversal). Callable independently for testing.	authenticated
/api/lead-time/defaults	GET	Get org-level lead time defaults	authenticated
/api/lead-time/defaults	PUT	Update org-level lead time defaults	admin

10.2 Modified Endpoints

Endpoint	Change	Change ID
GET /api/fulfillment-routes	Return route_category in list/detail DTOs	C1
POST /api/fulfillment-routes	Accept route_category in create DTO	C1
PUT /api/fulfillment-routes/{id}	Accept route_category in update DTO (blocked if route is in use)	C1
GET /api/product-variants/{id}	Return lead time component fields + resolved totals	C3
PUT /api/product-variants/{id}	Accept lead time component fields	C3
GET /api/inquiry-lines, /api/quotation-lines, /api/sale-order-lines	Return calculated_lead_time_days, calculated_delivery_date, manual_delivery_date, effective_delivery_date	C4
PUT /api/sale-order-lines/{id}	Accept manual_delivery_date override	C4
POST /api/sale-orders/{id}/confirm	Split behavior: stock lines → delivery, manufacture lines → production order	C5
GET /api/sale-orders/{id}/fulfillment	Return both delivery orders AND production orders in fulfillment view	C5, C6
GET /api/production-orders/{id}	Return fulfillment_route_id, delivery_order_id in response DTO	C6

10.3 New DTO Structures

csharp

```
// POST /api/lead-time/calculate
public class LeadTimeCalculateRequest
{
    public long VariantId { get; set; }
    public decimal Quantity { get; set; }
    public DateTimeOffset? RequestedDate { get; set; }
}

public class LeadTimeCalculateResponse
{
    public int TotalLeadTimeDays { get; set; }
    public DateTimeOffset EarliestDeliveryDate { get; set; }
    public List<LeadTimeComponentDto> Components { get; set; }
}

public class LeadTimeComponentDto
{
    public string Name { get; set; }
    public int Days { get; set; }
    public string Source { get; set; } // "BOM Calculation", "Manual", "Org Default", "Su
}
```



11. UI Wireframes & Specifications

11.1 Product Variant — Lead Times Tab (C3)

Product Variant: Custom Gear Assembly - Type A

[General] [Classification] [Stock] [Lead Times] [BOM]

▲ active

Route: Manufacture → Pick, Pack & Ship (MANUFACTURE)

Lead Time Components

Component	Days	Source
Manufacturing Lead Time	[5]	🕒 From BOM calc
Manufacturing Buffer	[2]	Variant override
Quality Inspection	[1]	Org default
Internal Transfer	[0]	Org default
Pick & Pack	[1]	Org default
Shipping Lead Time	[3]	Org default
Sales Safety Buffer	[1]	Org default
TOTAL LEAD TIME 13 days		

Source legend:

🕒 From BOM calc = calculated from BOM operations

Variant override = manually set on this variant

Org default = from Settings → Lead Time Defaults

[Recalculate from BOM]

[Reset to Org Defaults]

11.2 Sale Order Lines — Lead Time Button (C4)

Sale Order: SO-2001

Status: DRAFT

[Header] [Lines] [Fulfillment] [Attachments] [History]

▲ active

	Product	Qty	Price	Route	Delivery
1	Steel Pipes	500	\$450	[Pick&Ship ▼]	Oct 08
2	Custom Gear Assy	50	\$1200	[MFG P&S ▼]	– []
3	Copper Fittings	200	\$320	[PPS ▼]	Oct 10

= Calculated | = Manual override | [] = Click to calculate

Lead Time Breakdown (Line 2: Custom Gear Assembly)

Manufacturing Lead Time	5 days	(From BOM calculation)
Manufacturing Buffer	2 days	(Variant override)
Quality Inspection	1 day	(Org default)
Pick & Pack	1 day	(Org default)
Shipping Lead Time	3 days	(Org default)
Sales Safety Buffer	1 day	(Org default)

Total: 13 days → Earliest delivery: Oct 16, 2026

[Apply] [Override Date]

11.3 SO Fulfillment Tab — Mixed Order (C5, C6)

Sale Order: SO-2001

Status: CONFIRMED

[Header] [Lines] [Fulfillment] [Attachments] [History]

▲ active

— Delivery Orders (2) —

Delivery	Route	Lines	Status	Action
DO-20261003-001	Pick & Ship	1	● Released	[View]
└ Steel Pipes × 500				
DO-20261003-002	Pick Pack	1	● Draft	[View]
└ Copper Fittings × 200				

— Production Orders (1) —

Production	Product	Status	Delivery
PROD-085	Custom Gear Assy	● InProg	Pending
└ Route after production: Pick, Pack & Ship			
└ Delivery will be created when production completes			

Overall Fulfillment: 0 of 3 lines delivered

0%

Timeline:

Oct 03

SO Confirmed

Oct 03

DO-001, DO-002 created (stock lines)

Oct 03

PROD-085 created (manufacturing line)

Oct 16

Expected: PROD-085 completion → delivery auto-created

11.4 Production Order — Delivery Link (C6)

Production Order: PROD-085		Status: ● Completed
Product: Custom Gear Assembly		Variant: Type A
Sale Order: SO-2001 (line 2)		← clickable link
Fulfillment Route: Manufacture → Pick, Pack & Ship		
Production Summary		
Planned: 50 units		Actual: 48 units Yield: 96%
— Auto-Created Delivery —		
DO-20261016-005		Pick, Pack & Ship 48 units ● Draft
Created at FGR completion on Oct 16, 2026		
Note: Shortfall of 2 units (planned 50, produced 48)		
		[View Delivery]
[Overview] [Material Availability] [Issues] [Timeline]		

11.5 Lead Time Defaults — Settings Page (C3)

Settings → Lead Time Defaults

These defaults apply to all products that don't have variant-level overrides.

Pick & Pack Time	[1] days
Shipping Lead Time	[3] days
Sales Safety Buffer	[1] days
Manufacturing Buffer	[0] days
Quality Inspection Time	[0] days
Internal Transfer Time	[0] days

[Save]

Note: Supplier lead time and manufacturing lead time are always set at the product/variant level – they vary too much across products for a meaningful org default.

12. Business Rules

Rule ID	Change	Rule Description	Enforcement Point
BR-C1-01	C1	route_category must be one of: STOCK, MANUFACTURE, BUY, DROPSHIP	FulfillmentRouteService
BR-C1-02	C1	route_category cannot be changed on a route that is assigned to active product variants or open SO lines	FulfillmentRouteService
BR-C1-03	C1	BUY and DROPSHIP categories are reserved — routes with these categories cannot be assigned until future addendum activates them	FulfillmentRouteService
BR-C1-04	C1	Manufacturing seed routes (MFG_PICK_SHIP, MFG_PICK_PACK_SHIP) are system routes — cannot be deleted	TenantProvisioningService
BR-C2-01	C2	BOM tab is visible ONLY when product variant's route has route_category = 'MANUFACTURE'	Product UI
BR-C2-02	C2	Products with route_category = 'MANUFACTURE' MUST have at least one active BOM before use on a Sale Order	ProductVariantService
BR-C2-03	C2	is_manufacturable flag is auto-synced from route_category on route assignment change	ProductVariantService
BR-C2-04	C2	Changing route from MANUFACTURE to STOCK does not delete existing BOMs — only hides the tab	Product UI
BR-C3-01	C3	Lead time component values must be non-negative integers (≥ 0)	ProductVariantService, LeadTimeDefaultsService
BR-C3-02	C3	NULL on a variant lead time component means "use org default"	LeadTimeResolver

Rule ID	Change	Rule Description	Enforcement Point
BR-C3-03	C3	One LeadTimeDefaults row per org, auto-created on tenant provisioning	TenantProvisioningService
BR-C3-04	C3	Manufacturing-specific components (mfg lead time, mfg buffer) are hidden when route_category ≠ MANUFACTURE	Lead Times tab UI
BR-C3-05	C3	Shipping lead time component is hidden when route does not include SHIP step	Lead Times tab UI
BR-C4-01	C4	Lead time calculation is on-demand (button click), not automatic on product selection	Inquiry/Quotation/SO line UI
BR-C4-02	C4	BOM-aware calculation traverses the BOM tree recursively with cycle detection	LeadTimeCalculator
BR-C4-03	C4	Sub-components are treated as parallel (max), this-level operations as sequential (sum)	LeadTimeCalculator
BR-C4-04	C4	Stock-aware: if a sub-component has sufficient stock, its lead time contribution is 0	LeadTimeCalculator
BR-C4-05	C4	manual_delivery_date overrides calculated_delivery_date on effective_delivery_date	COALESCE computed column
BR-C4-06	C4	Lead time results are cached for 5 minutes per (variant_id, quantity_bucket)	LeadTimeCalculator
BR-C5-01	C5	SO confirmation splits lines by route_category: STOCK → Delivery, MANUFACTURE → Production Order	SaleOrderService.ConfirmAsync

Rule ID	Change	Rule Description	Enforcement Point
BR-C5-02	C5	Manufacturing lines do NOT create Delivery Orders at SO confirmation — delivery is deferred	SaleOrderService.ConfirmAsync
BR-C5-03	C5	Production Order created from SO carries sale_order_id, sale_order_line_id, and fulfillment_route_id	SaleOrderProductionCreator
BR-C5-04	C5	Production Order from SO triggers BOM explosion immediately after creation	SaleOrderProductionCreator
BR-C5-05	C5	Mixed orders (stock + manufacturing lines) process both types at confirmation	SaleOrderService.ConfirmAsync
BR-C5-06	C5	Production Order planned_start_date is backward-scheduled from SO line's effective_delivery_date minus post-production lead time	SaleOrderProductionCreator
BR-C6-01	C6	FGR confirmed on SO-linked PO → auto-create Delivery Order	FGRConfirmedEventHandler
BR-C6-02	C6	FGR confirmed on standalone PO (sale_order_id IS NULL) → NO auto-delivery; FG goes to inventory	FGRConfirmedEventHandler
BR-C6-03	C6	Auto-created delivery uses fulfillment_route_id from the Production Order	ProductionDeliveryCreator
BR-C6-04	C6	Delivery quantity = PO.actual_qty (not planned_qty) — reflects actual production yield	ProductionDeliveryCreator

Rule ID	Change	Rule Description	Enforcement Point
BR-C6-05	C6	If actual_qty < planned_qty: delivery for actual, shortfall flagged on SO line, notification sent	ProductionDeliveryCreator
BR-C6-06	C6	If actual_qty = 0: no delivery created, error notification to production manager	ProductionDeliveryCreator
BR-C6-07	C6	Auto-created delivery status = DRAFT — requires explicit release for warehouse picking	ProductionDeliveryCreator
BR-C6-08	C6	PO.delivery_order_id is populated after delivery auto-creation for traceability	ProductionDeliveryCreator
BR-C6-09	C6	For MTS escape hatch: salesperson overrides manufacturing route to stock route on SO line → delivery created at SO confirmation from stock	SaleOrderService + EffectiveRouteResolver

13. Test Scenarios

Test ID	Change	Scenario	Expected Result
T-C1-01	C1	Create fulfillment route with category MANUFACTURE	Route created with route_category = 'MANUFACTURE'
T-C1-02	C1	Create route with category BUY	Rejected: BUY category not yet activated
T-C1-03	C1	Change route_category on route assigned to 5 variants	Rejected: route is in use
T-C1-04	C1	Tenant provisioning creates new org	5 seed routes: 3 STOCK + 2 MANUFACTURE
T-C2-01	C2	Assign MFG_PICK_PACK_SHIP route to variant	BOM tab visible, is_manufacturable = TRUE
T-C2-02	C2	Change variant route from MFG to PICK_AND_SHIP	BOM tab hidden, is_manufacturable = FALSE, existing BOMs preserved
T-C2-03	C2	Use manufacturing variant on SO without active BOM	Validation error: active BOM required
T-C2-04	C2	Filter product list by manufacturing products	Returns variants where route_category = 'MANUFACTURE'
T-C3-01	C3	View lead times tab for manufacturing variant	Shows all 8 components with values from variant + org defaults
T-C3-	C3	View lead times tab for stock variant (PICK_AND_SHIP)	Manufacturing components hidden; shows 5 applicable components

Test ID	Change	Scenario	Expected Result
02			
T-C3-03	C3	View lead times tab for PICK_ONLY route	Shipping component hidden (no SHIP step)
T-C3-04	C3	Set variant pick_pack_days = 2 (org default = 1)	Lead times tab shows 2 days with "Variant override" source
T-C3-05	C3	Clear variant pick_pack_days (set NULL)	Lead times tab shows 1 day with "Org default" source
T-C3-06	C3	Update org lead time defaults	Changes reflected on all variants using org defaults
T-C4-01	C4	Click 🕒 on SO line for stock variant	Popover shows supplier + pick/pack + shipping + buffer = total
T-C4-02	C4	Click 🕒 on SO line for manufacturing variant (BOM with 3 components, all in stock)	Manufacturing lead time from BOM operations, component lead = 0
T-C4-03	C4	Click 🕒 for manufacturing variant with BOM components out of stock	Component lead time includes supplier lead for procured parts
T-C4-04	C4	Click 🕒 for variant with nested BOM (sub-assembly requires production)	Recursive calculation: sub-assembly production + parent production
T-C4-05	C4	Circular BOM detection	Returns 0 for cycle-breaking node, logs warning
T-C4-	C4	Calculate, then manually override delivery date	effective_delivery_date = manual_delivery_date, indicator shows

Test ID	Change	Scenario	Expected Result
06			"Manual"
T-C4-07	C4	Clear manual override	effective_delivery_date reverts to calculated_delivery_date
T-C4-08	C4	Same calculation within 5 minutes	Cached result returned (no BOM re-traversal)
T-C5-01	C5	Confirm SO with all stock-route lines	Delivery orders created immediately (Addendum 33 behavior)
T-C5-02	C5	Confirm SO with all manufacturing-route lines	Production orders created, no delivery orders
T-C5-03	C5	Confirm SO with mixed lines (2 stock + 1 manufacturing)	2 delivery orders for stock lines, 1 production order for mfg line
T-C5-04	C5	Confirm SO with manufacturing line — check PO fields	PO has sale_order_id, sale_order_line_id, fulfillment_route_id, status = DRAFT
T-C5-05	C5	Confirm SO with manufacturing line — BOM explosion	BOM explosion triggered, PMRs created
T-C5-06	C5	SO Fulfillment tab after confirmation (mixed order)	Shows both delivery orders section and production orders section
T-C5-07	C5	Cancel SO with manufacturing line in production	If PO status < IN_PROGRESS: PO cancelled. If IN_PROGRESS: PO remains, warning shown.
T-C6-	C6	FGR confirmed on SO-linked PO (planned=100, actual=100)	Delivery order created for 100 units with correct route

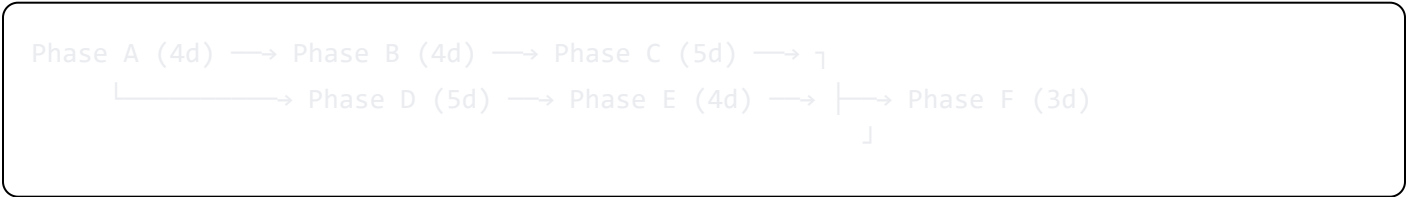
Test ID	Change	Scenario	Expected Result
01			
T-C6-02	C6	FGR confirmed on SO-linked PO (planned=100, actual=95)	Delivery for 95 units, shortfall notification, SO line flagged
T-C6-03	C6	FGR confirmed on SO-linked PO (planned=100, actual=0)	No delivery created, error notification
T-C6-04	C6	FGR confirmed on standalone PO (no SO link)	No delivery created, FG goes to inventory
T-C6-05	C6	Auto-created delivery has correct route steps	Delivery follows route's PICK → PACK → GOODS_ISSUE → SHIP
T-C6-06	C6	Auto-created delivery: from_source_type	from_source_type = 'SALE_ORDER'
T-C6-07	C6	PO.delivery_order_id populated after FGR	Link established for traceability
T-C6-08	C6	SO Fulfillment tab after production completes	Production order shows "Completed", delivery shows "Draft"
T-C6-09	C6	MTS escape hatch: override route on SO line to STOCK	SO confirmation creates delivery immediately from existing stock
T-C6-10	C6	End-to-end: SO confirm → PO → materials → production → FGR → delivery → pick → ship → complete	Full traceability chain verified

14. Development Phases

14.1 Phase Breakdown

Phase	Changes	Estimated Days	Dependencies
Phase A — Route Category & Classification	C1, C2 (route_category column, seed routes, BOM visibility logic, auto-sync flags)	4 days	Addendum 33 complete
Phase B — Lead Time Management	C3 (LeadTimeDefaults table, ProductVariants columns, Lead Times tab UI, org settings page)	4 days	Phase A (needs route_category for component visibility)
Phase C — Lead Time Calculation Service	C4 (ILeadTimeCalculator, BOM traversal, line-item columns, calculate button UI, popover)	5 days	Phase B (needs lead time schema)
Phase D — SO Confirmation Split	C5 (SaleOrderProductionCreator, modified ConfirmAsync, SO Fulfillment tab updates)	5 days	Phase A (needs route_category), Addendum 31 (PO creation)
Phase E — Production → Delivery Bridge	C6 (extended FGRConfirmedEventHandler, ProductionDeliveryCreator, PO UI updates)	4 days	Phase D (needs SO-linked POs)
Phase F — Testing & Integration	All changes — integration tests, E2E mixed-order scenario	3 days	Phase A-E
TOTAL		25 days	

14.2 Parallelization Opportunities



- **Phase B** and **Phase D** can run in parallel after Phase A completes
- Phase B → C is a sequential dependency (lead time schema before calculation service)

- Phase D → E is a sequential dependency (SO confirmation split before FGR delivery bridge)
- **With 2 parallel agents: critical path = A(4) + max(B+C, D+E)(9) + F(3) = 16 dev days**
- Calendar time with parallelization: **~16 working days**

14.3 New Permission Claims Summary

Claim Code	Module	Description
lead_time_defaults_manage	Settings	Create and update org-level lead time defaults

14.4 Risk Considerations

Risk	Impact	Mitigation
BOM traversal performance on deep BOM trees	Medium — recursive queries may be slow	5-minute cache per (variant, qty_bucket). Monitor query times. Add max depth guard (default 10 levels).
Mixed order complexity	Medium — stock + manufacturing lines on same SO	Clear separation in SaleOrderService.ConfirmAsync. Fulfillment tab shows both sections.
MTS vs MTO ambiguity for manufacturing products	Low — user may not know to override route	Delivery Preview on SO shows "Production Order will be created" for manufacturing lines. Clear messaging.
Partial production yield	Low — delivery qty < SO line qty	Shortfall notification + SO line annotation. Future addendum may handle backorder production.
FGR event handler reliability	Medium — delivery creation must not fail silently	Wrap in try/catch, log errors, send failure notification. Idempotency check: don't create duplicate delivery if event replays.

Appendix A — Deferred to Future Addendums

Topic	Description
BOM Product Management & Procurement	Creating Purchase Orders for BOM components, supplier selection for raw materials, procurement-to-production flow
BUY Route Category	Products procured from suppliers — Purchase Order at SO confirmation, delivery after GRN
DROPSHIP Route Category	Products shipped directly from supplier to customer — PO with customer delivery address
Backorder from Production Shortfall	Auto-create second Production Order when actual_qty < planned_qty
Lead Time by Customer Region	Shipping lead time varying by customer's delivery region/zone
Lead Time Alerts	Notify salesperson when calculated delivery date exceeds customer's requested date
Replenishment Rules	Automatic route switching based on inventory levels (buy if stock < threshold, manufacture otherwise)

End of FSD Addendum 34 — Route Classification, Lead Time & Production-to-Delivery Integration